Année de Formation 2025/2026

M205 : Développer en back end
TP : Template Blade

Exercice 1 : Affichage de variables

1. Créez une route dans votre fichier `web.php` pour afficher une vue nommée `welcome`.
2. Dans votre contrôleur, envoyez un nom de variable `nom` avec la valeur de votre choix à la vue.
3. Dans votre fichier `welcome.blade.php`, affichez la variable `nom` en utilisant la syntaxe Blade.

Solution :

```php
//Route dans web.php
Route::get('/welcome', function () {
    return view('welcome', ['nom' => 'Jean']);
});
```

```php
<!-- welcome.blade.php -->
<h1>Bonjour, {{ $nom }} !</h1>
```

Exercice 2 : Directives conditionnelles

1. Créez une variable `isAdmin` dans votre contrôleur et affectez-lui la valeur `true` ou `false`.
2. Dans la vue, affichez un message spécial uniquement si `isAdmin` est vrai, sinon, affichez un message standard.

Solution :

```php
//Route dans web.php
Route::get('/admin', function () {
    return view('admin', ['isAdmin' => true]);
});
```

```php
<!-- admin.blade.php -->
@if($isAdmin)
    <h2>Bienvenue, administrateur !</h2>
@else
    <h2>Bienvenue, utilisateur.</h2>
@endif
```

Exercice 3 : Boucles avec Blade

1. Créez une variable `produits` qui est un tableau de noms de produits, comme `['Ordinateur', 'Téléphone', 'Tablette']`.
2. Dans votre vue, utilisez une boucle `@foreach` pour afficher chaque produit dans une liste.

Solution :

```php
//Route dans web.php
Route::get('/produits', function () {
    $produits = ['Ordinateur', 'Téléphone', 'Tablette'];
    return view('produits', compact('produits'));
});
```

```blade
<!-- produits.blade.php -->
<ul>
    @foreach($produits as $produit)
        <li>{{ $produit }}</li>
    @endforeach
</ul>
```

Exercice 4 : Gestion des layouts

1. Créez un fichier de layout `layouts/app.blade.php` qui inclut une section pour un `header`, une section pour le contenu principal, et un `footer`.
2. Dans une autre vue, utilisez `@extends` pour hériter de ce layout, puis remplissez la section `content` avec un message personnalisé.

Solution:

```blade
<!-- layouts/app.blade.php -->
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Mon Application</title>
</head>
<body>
    <header>
        <h1>Mon Application</h1>
    </header>
    <main>
        @yield('content')
    </main>
    <footer>
        <p>&copy; 2024 - Mon Application</p>
    </footer>
</body>
</html>
```

```blade
<!-- welcome.blade.php -->
@extends('layouts.app')

@section('content')
    <h2>Bienvenue sur notre application Laravel</h2>
    <p>Voici notre premier test de layout Blade.</p>
@endsection
```

Exercice 5 : Inclusion de vues partielles

1. Créez une vue partielle `partials/menu.blade.php` avec un menu de navigation.
2. Dans votre layout principal (`app.blade.php`), incluez cette vue partielle dans le `header` en utilisant la directive `@include`.

Solution :

```blade
<!-- partials/menu.blade.php -->
<nav>
    <ul>
        <li><a href="/">Accueil</a></li>
        <li><a href="/produits">Produits</a></li>
        <li><a href="/contact">Contact</a></li>
    </ul>
</nav>
```

```blade
<!-- layouts/app.blade.php -->
<header>
    <h1>Mon Application</h1>
    @include('partials.menu')
</header>
```

Exercice 6 : Utilisation des Composants et des Slots

1. Créez un composant Blade pour une carte (`card.blade.php`) qui contient un `titre` et un `contenu`.
2. Dans le fichier Blade principal, utilisez le composant `card` et remplissez les slots `title` et `content` avec des valeurs de votre choix.

Solution :

```blade
<!-- resources/views/components/card.blade.php -->
<div class="card">
    <h3>{{ $titre }}</h3>
    <div>{{ $slot }}</div>
</div>
```

```blade
<!-- Dans votre vue principale, par exemple home.blade.php -->
<x-card :titre="'Mon Titre de Carte'">
    Ceci est le contenu de la carte.
</x-card>
```

Exercice 7 : Héritage avec Sections Multiples

1. Ajoutez une nouvelle section `sidebar` dans votre layout `app.blade.php`.
2. Dans votre vue principale, définissez le contenu de cette `sidebar` en utilisant `@section`.

Solution :

```
<!-- layouts/app.blade.php -->
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Mon Application</title>
</head>
<body>
    <header>
        <h1>Mon Application</h1>
    </header>
    <aside>
        @yield('sidebar')
    </aside>
    <main>
        @yield('content')
    </main>
    <footer>
        <p>&copy; 2024 - Mon Application</p>
    </footer>
</body>
</html>
```

```
<!-- accueil.blade.php -->
@extends('layouts.app')

@section('content')
    <h2>Contenu Principal</h2>
    <p>Ceci est le contenu principal de la page.</p>
@endsection

@section('sidebar')
    <h3>Barre latérale</h3>
    <p>Liens vers les ressources et les sections connexes.</p>
@endsection
```

Exercice 8 : Boucles avec Index et Directive `@empty`

1. Créez une liste de catégories de produits (`categories`) et passez-la à la vue.
2. Dans la vue, utilisez une boucle `@foreach` pour afficher chaque catégorie avec son index.
3. Utilisez la directive `@empty` pour afficher un message si la liste est vide.

Solution :

```php
//web.php
Route::get('/categories', function () {
    $categories = ['Electronique', 'Vêtements', 'Livres'];
    return view('categories', compact('categories'));
});
```

```blade
<!-- categories.blade.php -->
<ul>
    @forelse($categories as $index => $categorie)
        <li>{{ $index + 1 }}. {{ $categorie }}</li>
    @empty
        <p>Aucune catégorie disponible.</p>
    @endforelse
</ul>
```

Exercice 9 : Création d'une Directive Personnalisée pour le Formatage de Texte

1. Dans le fichier `AppServiceProvider`, créez une directive Blade personnalisée nommée `@caps` pour mettre un texte en majuscules.
2. Utilisez `@caps` dans une vue pour mettre un texte en majuscules.

Solution :

```php
//AppServiceProvider.php
use Illuminate\Support\Facades\Blade;

public function boot()
{
    Blade::directive('caps', function ($expression) {
        return "<?php echo strtoupper($expression); ?>";
    });
}
```

```blade
<!-- dans votre vue Blade -->
@caps('texte en minuscules')
```

Exercice 10 : Inclusion Conditionnelle d'une Vue Partielle

1. Créez une vue partielle `footer.blade.php` avec un pied de page.
2. Dans votre vue principale, utilisez une condition pour inclure le `footer` uniquement si une variable `afficherFooter` est définie sur `true`.

Solution :

```blade
<!-- partials/footer.blade.php -->
<footer>
    <p>&copy; 2024 - Mon Application</p>
</footer>
```

```blade
<!-- Dans la vue principale, par exemple accueil.blade.php -->
@if($afficherFooter)
    @include('partials.footer')
@endif
```

Exercice 11 : Création d'un Composant avec des Attributs Dynamiques

1. Créez un composant `button.blade.php` qui accepte des attributs dynamiques (`class`, `type`, etc.).
2. Utilisez le composant `button` dans une vue et passez-lui une classe et un type spécifiques.

Solution:

```blade
<!-- resources/views/components/button.blade.php -->
<button {{ $attributes->merge(['class' => 'btn']) }}>
    {{ $slot }}
</button>
```

```blade
<!-- Dans votre vue principale -->
<x-button class="btn-primary" type="submit">Soumettre</x-button>
```

Exercice 12 : Démonstration de la Directive `@once`

1. Utilisez la directive `@once` pour inclure un message unique, par exemple une alerte de bienvenue, qui ne s'affichera qu'une seule fois même si elle est dans une boucle.

Solution :

```blade
<!-- welcome.blade.php -->
@foreach(range(1, 5) as $i)
    @once
        <p>Bienvenue ! Ce message s'affiche une seule fois.</p>
    @endonce
    <p>Numéro {{ $i }}</p>
@endforeach
```